
Jac LLM Weekly — Qwen3-Coder 30B-A3B

Model card, sample outputs, and the `jac check` / `jac test` numbers

Ayush · Jaseci Labs · September 16, 2026

Experiment 08-nitin-new2-ds · Apple Silicon · MLX

Qwen3 Coder — the model card, corrected

Every line checked against the run's own config and logs

Base

- Qwen3-Coder-30B-A3B-**Instruct**, MoE: 48 blocks, 128 experts, top-8 routing, ~3B active
- **4-bit MLX** (group 64); the routers (mlp.gate) are kept at **8-bit**

Spectrum

- **16 of 48** blocks: [0, 22, 23, 27, 30, 34, 36-39, 41-45, 47]
- Marchenko–Pastur bulk-edge SNR, z -scored per module type, top-16 by mean
- **Reused from 04/06/07** — sha256-identical; the scan was *not* re-run for 08

Target — 8 modules per selected block

- q/k/v/o_proj, mlp.gate (the router)
- switch_mlp.{up,down,gate}_proj — the *stacked expert* projections, all 128 at once
- 256 LoRA tensors, 48 LoRASwitchLinear; all 256 keys verified present at load
- Note: the SNR *ranking* used only q/k/v/o + router, never the experts

LoRA & schedule

- rank **16**, scale **2.0**, dropout **0.05** (mlx-lm has *scale*, not *alpha*)
- **281.8M** trainable = **0.92%** of 30,532M
- Adam, LR 2×10^{-5} cosine decay, warmup 820
- batch 1, 8200 iters, max_seq_length 3072, mask_prompt: `true`
- **Epoch ≈ 0.65** — 8200 iters $\div$ 12,570 rows. Derived; the repo never writes an epoch count.

Outcome

- train loss 0.083, val loss 0.048 (from val 0.890 at iter 1)
- 2,494,898 trained tokens, peak 28.2 GB, ~4.3–5 h compute

Correction: the slide says CPT $\gg$ SFT. **08 ran SFT only** — CPT is out of scope for this lineage (CPT-v2 was rejected in phase 03) and both DPO arms were cut on 2026-09-11. CPT $\gg$ SFT describes the earlier 04-cpt-sft run.

Toolchain: these numbers are jaxlang 0.16.1, not the deck's “fixed 0.36.X”. A 50-task subset re-graded on the standalone jac 0.36.1 binary agrees within 5pp.

Model Output — Basic Functions

“write fibonacci in jac. def fib(n:int) -> list[int] {”

Base (no adapter)

jac check: FAIL

```
def fib(n: int) -> list[int]:
    if n <= 0:
        return []
    elif n == 1:
        return [0]
    elif n == 2:
        return [0, 1]

    result = [0, 1]
    for i in range(2, n):
        result.append(result[i-1] + result[i-2])

    return result
```

E0005 Unexpected token 'if' at 2:5, then E0005 on every return. It writes **Python** — colons, indentation, elif, no braces, no semicolons — while its own prose says “in JAC”.

08 adapter

jac check: PASS

jac run: PASS

```
def fib(n: int) -> list[int] {
    if n <= 0 {
        return [];
    }
    if n == 1 {
        return [0];
    }
    if n == 2 {
        return [0, 1];
    }
    result = [0, 1];
    for i in range(2, n) {
        result.append(result[i - 1] + result[i - 2]);
    }
    return result;
}
```

Braces, semicolons, typed signature, list built and returned correctly for every n . **The deck's note on this prompt** — “logic looks correct, but it did not use list properly” — **does not apply to this arm**. (That note appears verbatim on both the 30B and the 9B slide, so it is a placeholder; treat it as unattributed.)

Greedy, temp 0, single turn, 1536 max tokens. eval/gen_showcase.py → results/showcase/.

Model Output — Front-end

“can you write a personal webpage for me as a jsx component in jac?”

Base no Jac at all

- 6,198 characters in a ```jsx fence, opening with `import React, { useState }.` No Jac anywhere.

08 adapter right shape, runs away

- Opens ```jac, writes `def:pub PersonalWebpage() -> JsxElement, Tailwind className, header / hero / about / skills.`
- At our 1536-token cap it ran off the end mid-element. **Re-run at 4096** it finishes: **16,076 characters**, fence closed — and **still fails jac check** (E0002 Missing `;`).
- So it is *not* a truncation artifact. An open prompt makes it write a **very large page** that it cannot keep syntactically consistent to the end.

So we **guided** it — name the file, name the top-level component, name the construct to use, and forbid JavaScript:

“Write a personal webpage as a Jac client component in a file named `profile.cl.jac`. Top-level component must be `def:pub app() -> JsxElement` returning JSX. Show a name, a short bio, and a list of links built with a `{for ...}` statement slot. Use `className` with Tailwind classes. Do not write React or JavaScript syntax.”

Result: 1,818 characters — **9× shorter** — closed fence, correct component signature, correct statement-slot shape, and **4 compile errors** instead of an unbounded page. **Guiding the prompt shrinks the blast radius; it does not fix the semantics.** Next slide.

Front-end, guided — what still breaks

It knows the shape of a Jac component. It does not know it is not JavaScript.

```
def:pub app() → JsxElement {
  <div className="min-h-screen bg-gradient-to-br ... p-6">
    ...
    <h2 className="text-lg font-semibold">Connect with me</h2>
    {for link in [
      {"label": "GitHub", "url": "https://github.com"},
      {"label": "LinkedIn", "url": "https://linkedin.com"}
    ] {
      <a href={link.url} className="block py-2 px-4 ...">
        {link.label}
      </a>
    }}
    ...
  </div>
}
```

jac check — 4 errors, 3 warnings

- **E1004** 'app' declared return type JsxElement but may implicitly return None — **there is no return**. The whole JSX tree is a bare expression statement.
- **E0002** Missing ';' — same root cause.
- **E1030** dict[...] has no attribute "url" — and again for "label". link.url is JavaScript; Jac wants link["url"].
- **W2021** 'for' loop in a JSX slot renders JSX without a 'key'.

What it got right: def:pub app() -> JsxElement, the {for ... { ... }} statement slot (not .map()), className, Tailwind, and no stray import React.

Syntax: learned. Semantics: not yet. And the reason is not the model — next slide.

Front-end — the model is copying its training data

We audited the corpus with the same `jac` check we grade the model with

Stratified `jac` check over the **gold answers** in 08's SFT train split (302 rows, 60 per cell, seed 42):

Task type	Category	Rows	Gold compiles
code_gen	Graph	1,366	98.3%
code_gen	OSP	803	98.3%
js2jac	Function	2,871	86.7%
py2jac	Function	3,061	73.3%
js2jac	Fullstack	4,467	53.3%
population-weighted		12,570	≈73.6%

`eval/audit_corpus.py`; raw verdicts in `results/jac_check/corpus_audit.jsonl`.

The `JsxElement` rows specifically

- 3,080 train rows declare `-> JsxElement`.
- **1,404 of them (45.6%) contain no return in the component body.**
- Of a 40-row sample of those: **2/40 compile**. Top errors `E0002` ×37, `E1004` ×36.
- Of a 40-row sample *with* a return: **31/40 compile**.

`E0002` + `E1004` is exactly what the model emitted. It did not invent the bug, it learned it — the `js2jac` translator drops the return when it lifts a JS arrow-function component.

Cheap fix: gate every released row on `jac` check; that removes ~2,100 broken rows from the corpus's largest cell.

Graph & OSP — the over-fitting probe first

“write in python a dfs search over a tree” — does a Jac-tuned model still speak Python?

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def dfs_recursive(root, target):
    """Recursive DFS search for a target value in a binary tree."""
    if not root:
        return False
    if root.val == target:
        return True
    return dfs_recursive(root.left, target) or \
           dfs_recursive(root.right, target)
...
```

08 adapter

[followed the instruction](#)

- Answers in a ```python fence, 2,845 chars: recursive DFS, iterative DFS, path-returning DFS, plus a runnable `__main__` demo.
- **No Jac leakage.** 0.65 epochs of Jac-only SFT did not overwrite the base model's Python.
- The base model answers the same way — so this costs nothing and confirms nothing was lost.

Worth keeping as a regression check, but it does not exercise OSP — so we re-ran it with a prompt that does.

Graph & OSP — the real prompt

“write a dfs search with jac nodes and walker, create data in with entry {} block”

Base **answers in JavaScript**

```
// DFS Search with Jac Nodes and Walker
class JacNode {
  constructor(value) {
    this.value = value;
    this.children = [];
    this.visited = false;
  }
  addChild(child) { this.children.push(child); }
}

class Walker {
  dfsTraversal() {
    const stack = [this.currentNode];
    ...
  }
}
```

Asked for “jac nodes and walker”, the base model writes a ```javascript fence with class JacNode and class Walker. It has the vocabulary and none of the language.

08 adapter **jac check: PASS** **jac run: PASS**

```
node Node {
  has value: int = 0;
}

walker DFS {
  has visited: list[int] = [];

  can start with Root entry {
    visit [-->];
  }

  can traverse with Node entry {
    if here.value not in self.visited {
      self.visited.append(here.value);
      print(f"Visiting node {here.value}");
      visit [-->];
    }
  }
}

with entry {
  root ++> Node(value=1)
  ++> Node(value=2)
  ++> Node(value=3)
  ++> Node(value=4)
  ++> Node(value=5)
  ++> Node(value=6);
  root spawn DFS();
}
```

4.0s, 754 characters. Real archetypes, real can ... with ... entry abilities, real visit/spawn/++>. Printed output: Visiting node 1 ...6. Looks right.

Graph & OSP — the failed case

It compiles, it runs, the output looks correct, and it is wrong twice

Defect 1 — the `with entry` builds a chain, not a tree.

The indentation reads as a tree. Chained `++>` is left-associative and connects *sequentially*, so the indentation is cosmetic. Walking the real edges:



Defect 2 — `visit [-->]` is breadth-first.

`visit` *appends* to the walker's queue. Re-running the model's own walker over a real tree gives 1, 2, 5, 3, 4, 6 — BFS. The prompt asked for DFS.

The two defects hide each other. On a chain, BFS and DFS emit the same order, so the printed Visiting node 1...6 looks like a correct depth-first walk. `jac` check passes. `jac` run passes. Only re-shaping the data exposes it.

Verified fix (compiles and runs; emits 1,2,3,4,5,6 on a real tree)

```
walker DFS {
  can start with Root entry { visit:0:[-->]; }
  can traverse with Node entry {
    print(f"Visiting node {here.value}");
    visit:0:[-->];      # :0: pushes -- append is BFS
  }
}

with entry {
  n1 = Node(value=1); n2 = Node(value=2); ...
  root ++> n1;
  n1 ++> n2; n1 ++> n5;    # one statement per edge
  n2 ++> n3; n2 ++> n4;    # → a real tree
  n5 ++> n6;
  root spawn DFS();
}
```

Why this matters for the metric. Both of our current gates — “does it compile” and “does it run” — score this sample as a **pass**. Only a hidden behavioural test with a branching fixture catches it. This is the concrete argument for the `jac` test column on the next slides.

Statistics — Qwen3 30B-A3B — jac check

Sealed function eval, 1,000 tasks, greedy pass@1. Nitin's jac-data-gen v1 test split, zero overlap with training.

Task type	Category	has_block	jac check		lift over base
code_completion	Function	498/500 99.6%	296/500	59.2%	+47.8pp
py2jac (translation)	Function	500/500 100%	419/500	83.8%	+83.8pp
all	Function	998/1000 99.8%	715/1000	71.5%	+65.8pp
base, no adapter	Function	871/1000 87.1%	57/1000	5.7%	—
reference solutions	Function	1000/1000	862/1000	86.2%	ceiling

- **has_block is no longer the bottleneck:** the base loses 129 tasks to “never emitted a Jac fence”; 08 loses 2.
- The **86.2% reference ceiling** is not a typo — 138 of the eval's own gold solutions do not compile. Read 71.5% against 86.2%, not 100%.

- **Completion is the weak half:** given a prefix the model must close someone else's braces — 59.2% vs 83.8% for whole-file translation.
- Cross-check on jac 0.36.1 (50-task subset): 66% vs 71.5%. Same conclusion.

jac check by task type × knowledge category

08's own held-out validation split, 735 prompts, greedy pass@1 — and what the *gold answer* scores on the same rows

Task type	Category	n	08 jac check		gold ceiling	
code_completion [†]	Function	500	296	59.2%	431	86.2%
py2jac	Function	150	120	80.0%	120	80.0% tie
js2jac	Function	150	122	81.3%	138	92.0%
js2jac	Fullstack	150	84	56.0%	74	49.3% +6.7pp
code_gen	Graph	150	143	95.3%	149	99.3%
code_gen	OSP	135	95	70.4%	131	97.0%
all but code_completion		735	564	76.7%	612	83.3%

- **On Fullstack the model beats its own training targets** (56.0% vs 49.3%; 25 rows it gets right and the gold does not, 15 the other way). **51 of 150 rows neither side compiles** — that cell is ceiling-limited, not model-limited.
- py2jac is an **exact tie** with the gold (18 discordant pairs each way).
- **One real gap left:** code_gen/OSP **−26.7pp** against a 97% ceiling. Graph is only **−4.0pp** and js2jac/Function **−10.7pp**.

- **Corrected for our own generation cap.** All 20 rows that produced no fence at 1536 tokens were re-run at **4096**: 17 recovered, 14 of those compile. Table above is the corrected number; at 1536 it read 74.8% / 97.3%.
- has_block is then **732/735 = 99.6%**. The 3 that still emit no fence run past 13,000 characters — runaway, not refusal.
- Failure signature per cell: OSP → E1030 (uses an attribute the archetype never declares); Fullstack → E1032+E0002 (the corpus's own no-return JSX pattern); py2jac → the type checker (E1053/E1055/E1054); Graph → was mostly our token cap, now nearly clean.

[†] code_completion is the sealed function-eval row from the previous slide (a different, harder set, generated at that harness's own 1024-token budget — 08 lost only 2 of 1,000 tasks to it); the other five cells are 08's SFT validation split, stratified sample, seed 42, categories assigned by regex over the *gold* answer. Gold ceiling = the same jac check on the same rows' reference answers. **These are our holdouts, not Rui-Yu's — not cell-for-cell comparable to the Ornith table.**

jac check — the error classes

What the 283 compile failures actually are. First error[Exxxx] per failing answer, full re-run.

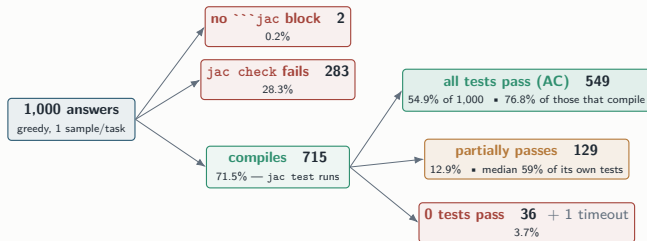
E0002	168	missing '}' 103 * ';' 59
E1001	22	cannot assign to this expression
E1053	12	argument type mismatch
E1002	11	cannot return this expression
E1004	11	may implicitly return None
E1055	10	no operator overload
E1054	10	no matching overload
E1032	7	type is unknown
other	22	

- **E0002 alone is 59% of all failures** — 103 of those a **missing closing brace**. The model loses count of its own nesting on long completions.
- Split by task: **completion fails on syntax** (166 of its 202 failures are E0002); **translation fails on the type checker** (E1001/E1053/E1054/E1055).
- The base model's 814 failures are E0005 Unexpected token ×458 — not malformed Jac, just Python.

Actionable: a brace-balance repair pass, or constrained decoding, converts a large slice of those 103 E0002 failures into gradeable answers for free — no retraining.

Statistics — Qwen3 30B-A3B — jac test

Where 1,000 generated answers end up. Every box is a disjoint bucket; percentages are of 1,000.



29,892 hidden test cases

18,817 pass 62.9%

11,075 fail

% AC = 54.9% (every hidden test in the task passes) ▪ % test cases passed = 62.9% ▪ base 2.9% / 4.5% ▪ reference ceiling 84.6% / 86.5%

By task: translation 82.2% AC, completion 27.6% AC. Almost the whole partial-pass band (123 of 129) is code_completion.

jac test — what “partially passes” means

129 answers compile, run, and pass some but not all of their hidden tests

Inside those 129 answers, **2,165 of 3,867** test blocks pass (56.0%); the median answer passes **59%** of its own tests. Re-running every block in isolation and classifying the 1,702 failures:

Why the block failed	blocks	answers
wrong value, ordinary input	1,326 78%	109 85%
missed edge case	153 9%	39 30%
runtime exception	181 11%	22 17%
runaway recursion	12 1%	1 1%
other	30 2%	—
hang / infinite loop	0	0

Exceptions seen: Value 59, Type 52, Key 35, Name 35, Index 11, Attribute 11, ZeroDivision 8. “Edge case” = the failing block’s input is degenerate — “, [], {}, (), 0, None, a negative, or a one-element container. Columns add past 129: one answer can fail several ways.

The characteristic partial pass — fn-complete-389801, **46 of 47 tests pass**:

```
"""Ensure that elements is a type of sequence, otherwise
it converts it to a list with only one element."""
def make_sequence(elements: list|set|tuple|None|int|float
                  |str|bool) → list|set|tuple {
    if isinstance(elements, (list, set, tuple)) {
        return elements;
    }
    return [elements];      # <- the model stops here
}
```

The one failing block is `assert make_sequence(None) == []`; — the model returns `[None]`. The reference has an arm the **docstring never mentions**: `elif elements is None { return []; }`.

No hangs, almost no exceptions. Partial pass is overwhelmingly a *specification* failure, not a robustness failure: the model writes plausible, terminating, type-correct code for the spec it was given, and loses on the clause nobody wrote down.

Where we are — output comparison

Same five prompts, base vs. 08 adapter, greedy, one turn

Prompt	Base	08 adapter	Verdict
<code>fib(n) -> list[int]</code>	Python syntax inside a <code>```jac</code> fence; E0005	braces, semicolons, typed, correct list	PASS
personal webpage, jsx	<code>```jsx + import React</code>	def:pub ... -> JsxElement, Tailwind, 16k chars at a 4096 budget	E0002
same, guided to <code>.cl.jac</code>	3-character reply (<code>```</code>)	right component shape, no return, <code>link.url</code> on a dict	4 errors
dfs in <i>python</i>	Python	Python, 3 variants + demo	PASS
dfs with jac nodes & walker	<code>class JacNode</code> in JavaScript	real archetypes; compiles <i>and</i> runs	BFS, chain

Settled

- Lexical/syntax **solved for functions and OSP**: 99.6–99.8% emit a Jac block, 71.5% compile on the sealed eval, 76.7% on our holdout, 83.8% on translation.
- No longer confuses Jac with Python or JavaScript — and did not lose Python.

Open, in priority order

- **Corpus hygiene first**: $\approx 26\%$ of our own SFT targets do not compile. On Fullstack the model already **beats** its targets (56.0% vs 49.3%) — that cell is ceiling-limited. Gate every row on jac check.
- **Semantics**: visit order, graph shape, unstated spec clauses. The real model gaps are `code_gen`/OSP (−26.7pp vs a 97% ceiling) and completion. Needs behavioural tests in training, not compile gates.
- **Brace accounting**: 103 tasks lost to one missing `}`.